Interface

What does the data structure represent?

A collection, a set, a sequence,
a map, a graph, the world, …

What operations does it support?

adding, removing, finding elements;
membership testing;
range searching; …

Implementation

Performance: speed and memory

How long does each operation take?

How much space does it use?

example

Sets (or sorted sets)	Binary search trees
Sequences	Skiplists
Maps	Hash tables
Graphs	Adjacency Lists

You can have different implementations of the same interface

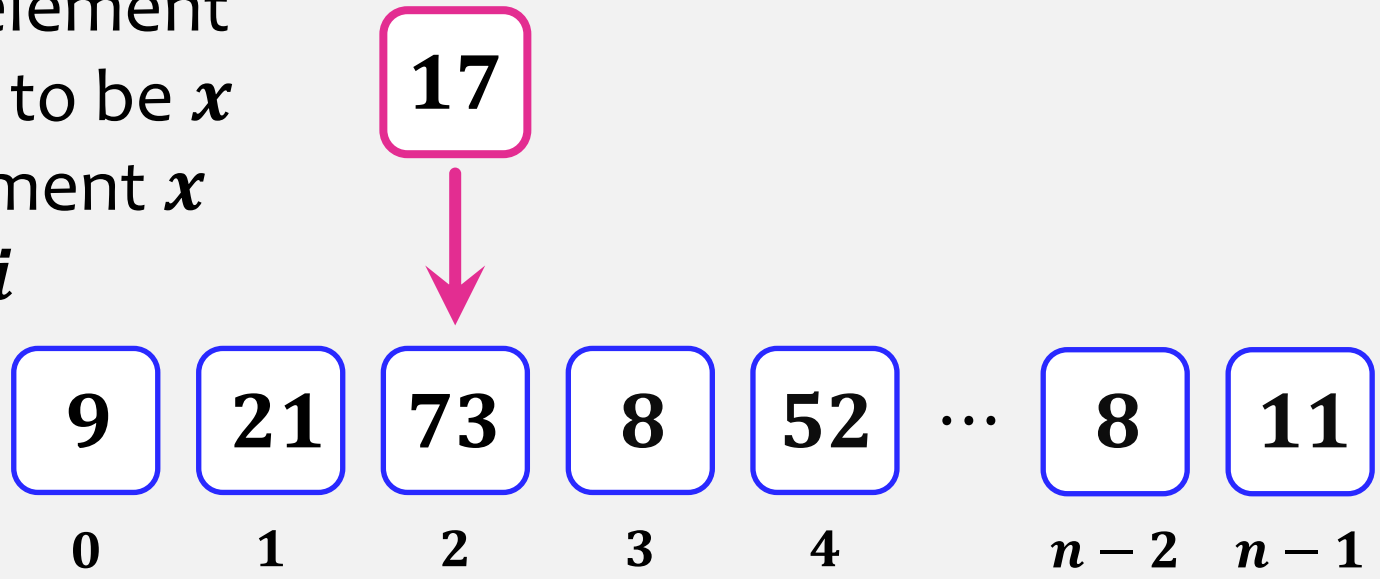
Abstract data types – List

Lists – store an indexed sequence of elements

Example: movie list(season 1,2 3,…)

Methods:

- size() – returns the number of elements on the list (n)
- get(i) – returns the element at position i
- set(i, x) – update the element at position i to be x
- add(i, x) – add the element x to position i



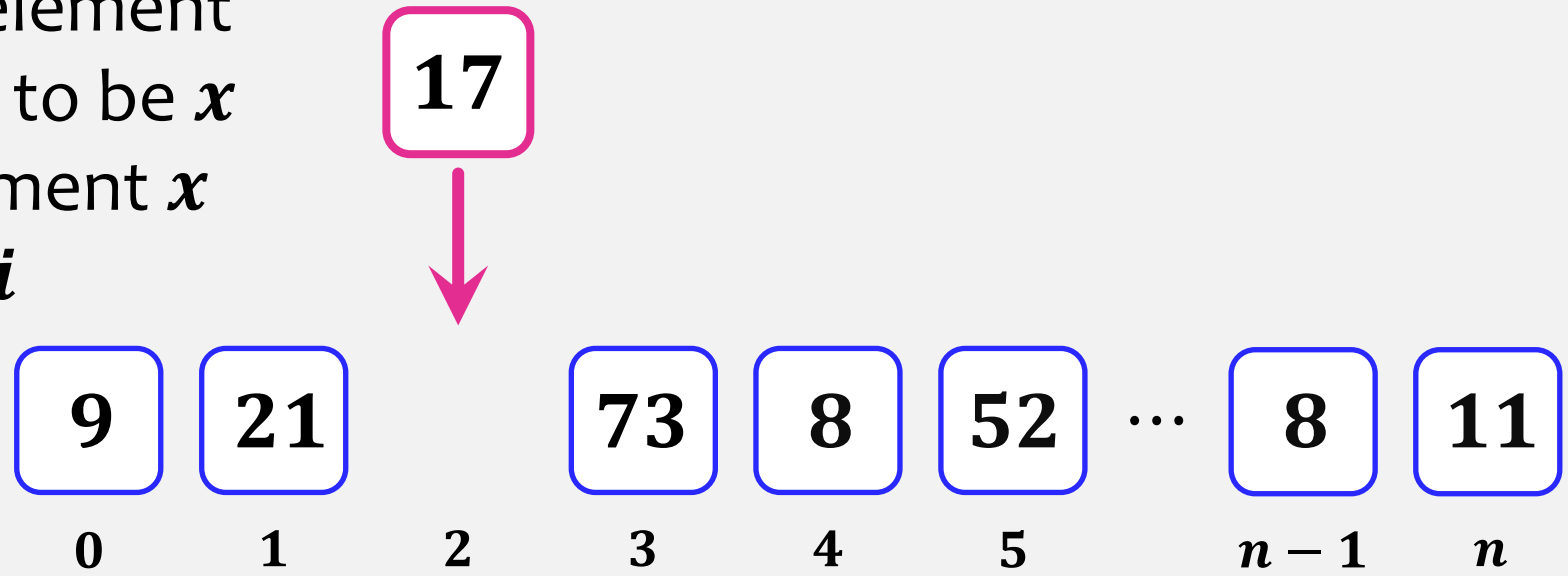
Abstract data types – List

Lists – store an indexed sequence of elements

Example: movie list(season 1,2 3,…)

Methods:

- size() – returns the number of elements on the list (n)
- get(i) – returns the element at position i
- set(i, x) – update the element at position i to be x
- add(i, x) – add the element x to position i
- remove(i) – remove element at position i

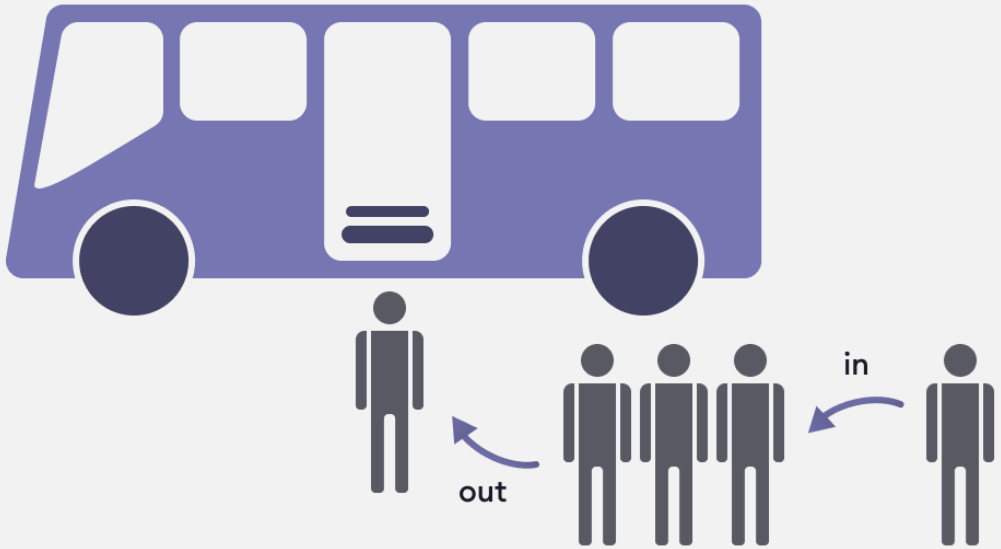


Abstract data types – Queue

Example: queue at the cash register

Queue – store elements that are accessed in first in, first out order (FIFO)
Not indexed. You can only access the oldest remaining element.

If we have **List** interface:
- add(size(),x)
- remove(0)



Methods:
- size()
- add(x) – add the element x to the end of the queue
- remove() – remove the first element of the queue

this element that has been in the queue the longest

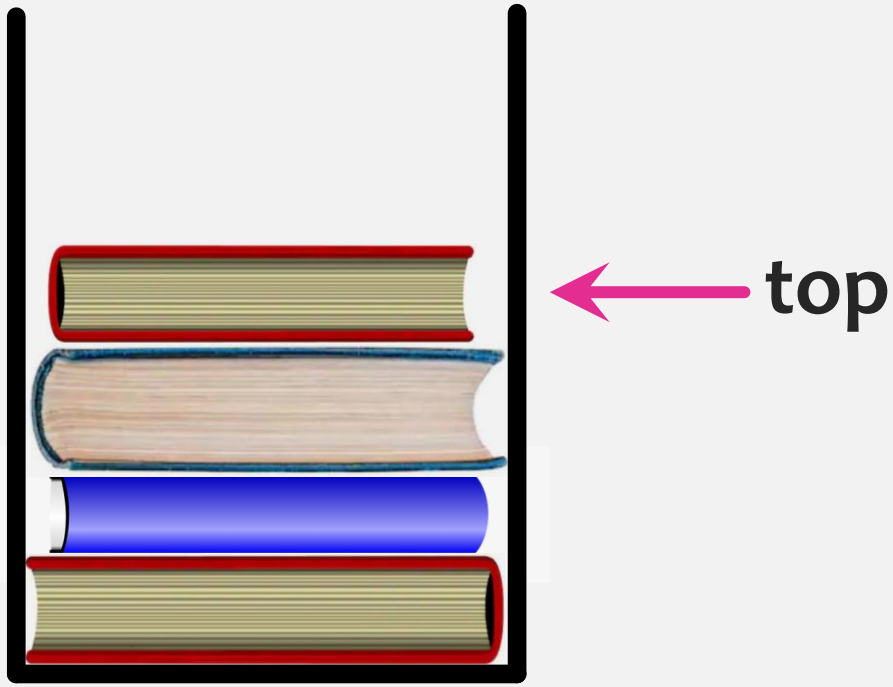
Abstract data types

Example: browser back button; stack of books or plates

Stack – store elements that are accessed in First-In-Last-Out order (FILO)

≡ Last-In-First-Out (LIFO);

No indices.



Methods:

- size()
- add(x) – add the element x to the top of the stack
- remove() – remove the element that was most recently added

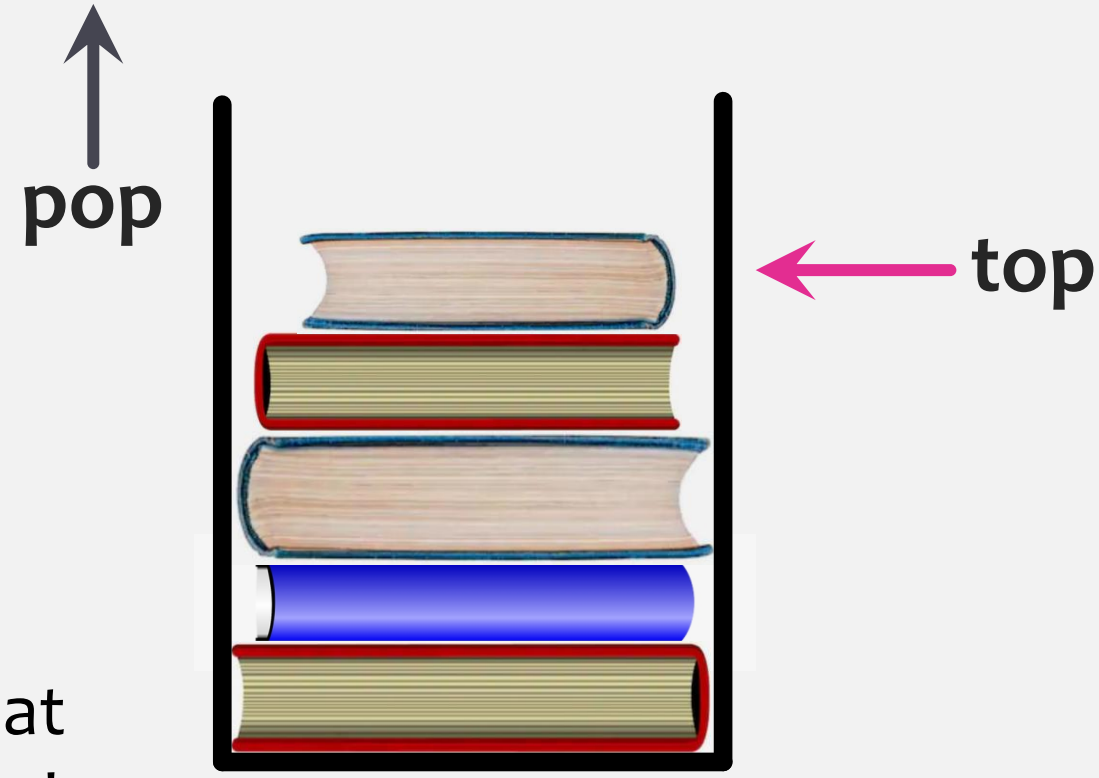
Abstract data types

Example: browser back button; stack of books or plates

Stack – store elements that are accessed in First-In-Last-Out order (FILO)

≡ Last-In-First-Out (LIFO);

No indices.



Methods:

- size()
- add(x) – add the element x to the top of the stack
- remove() – remove the element that was most recently added

Abstract data types

Priority Queue – stores elements that are accessed by min priority first.

Example: hospitals; VIP at clubs

Methods:

- size()
- add(x) – adds item with priority x
- remove() – removes the element with the lowest value/priority

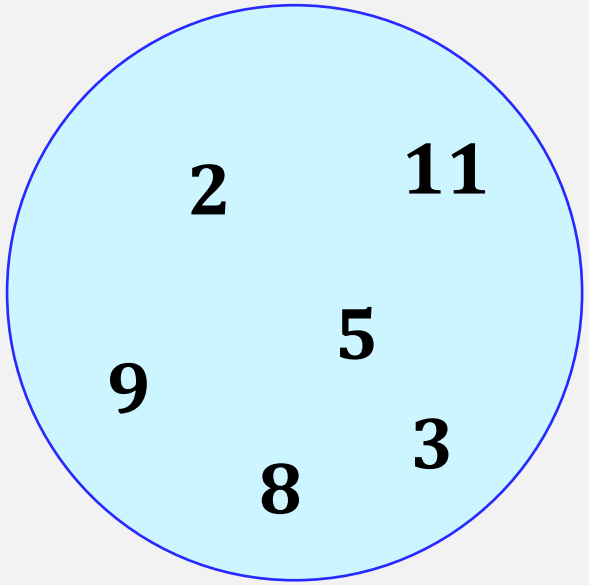
Abstract data types

Example: students in the class; contacts in my phone

Set – unordered collection of distinct elements

USet – unordered set

SSet – sorted set



Methods:

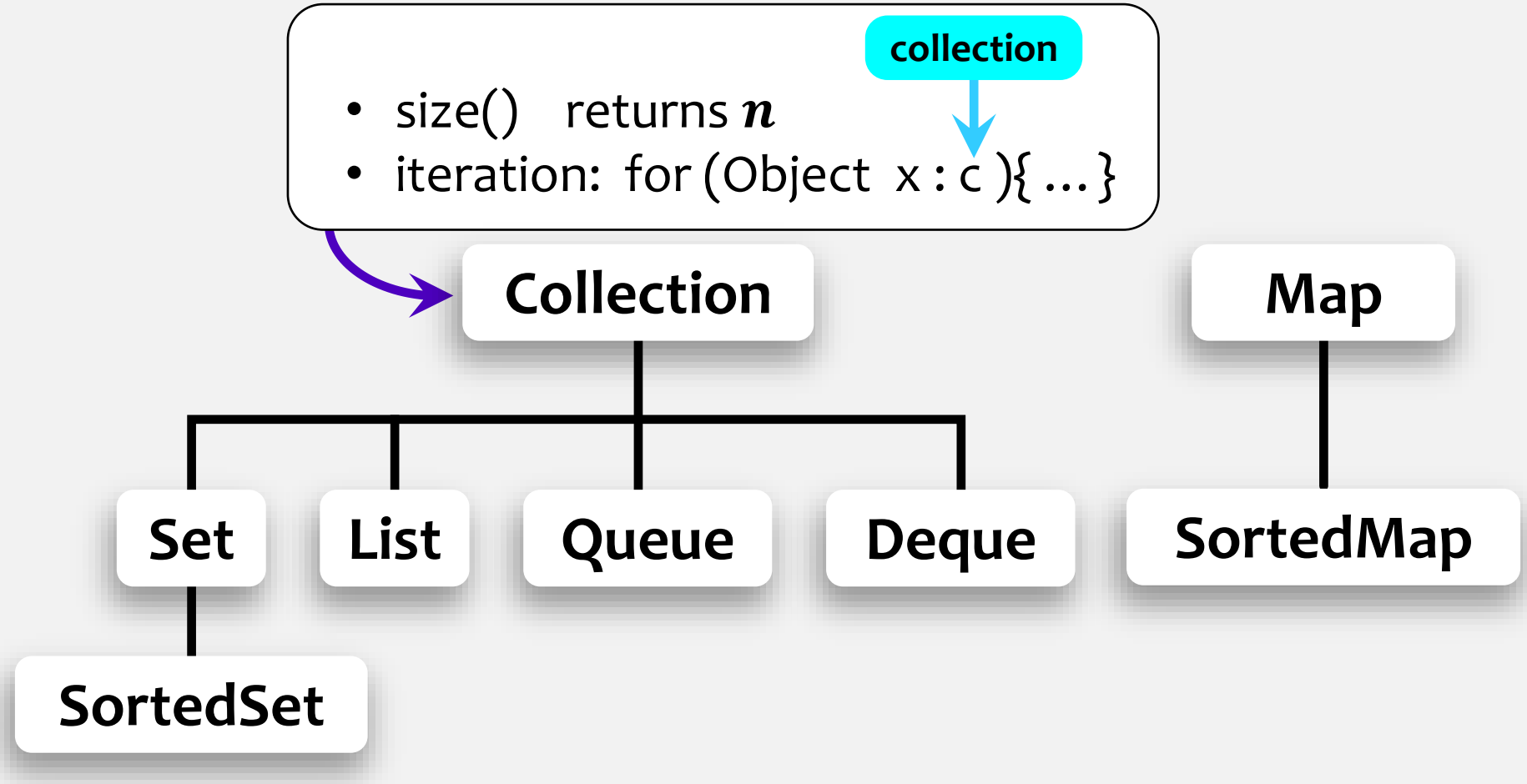
- size()
- add(x) – add the element x (if not already present)
- remove(x) – remove and return x. Return **null** if no such element x exists
- find(x) – generally returns whether x is in the set, but not quite

  USet – returns any y that is equivalent to x.

  If x is not in the set, returns **null**.

  SSet – if x is not in set, returns successor → **min** $y \geq x$

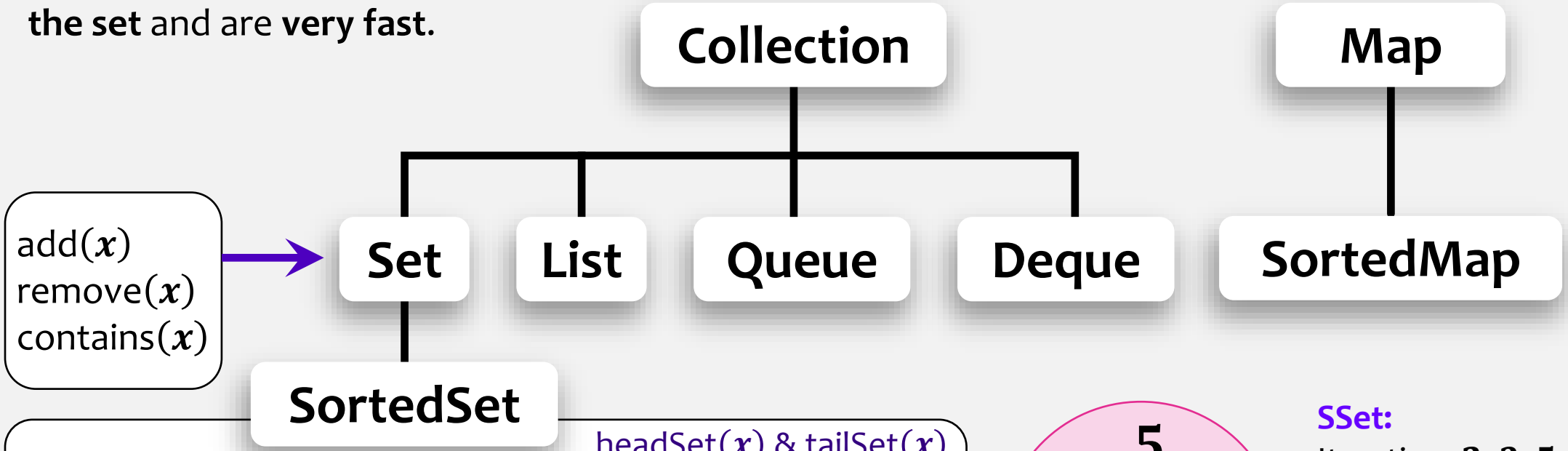
The Java Collections Framework: Interfaces



https://docs.oracle.com/javase/tutorial/collections/interfaces/index.html

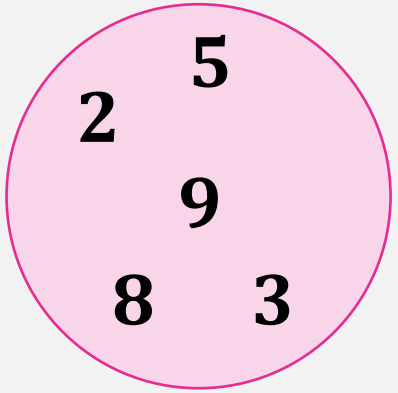
The Java Collections Framework: Interfaces

In a good set implementation, these operations on sets are **independent of the size of the set** and are **very fast**.



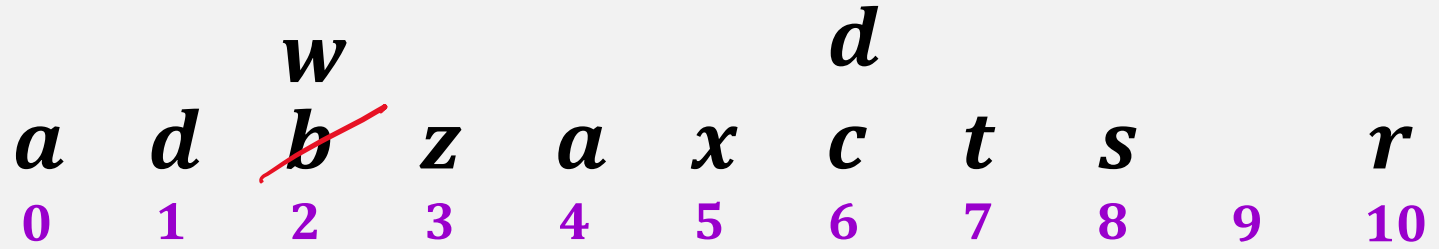
first() – smallest item in set
last() – largest item
headSet(x): returns everything smaller than x
tailSet(x): returns everything that is greater or equal to x
subSet(x, y): everything that is $\geq x$ and $< y$

headSet(x) & tailSet(x) return a view of the set; not a new copy



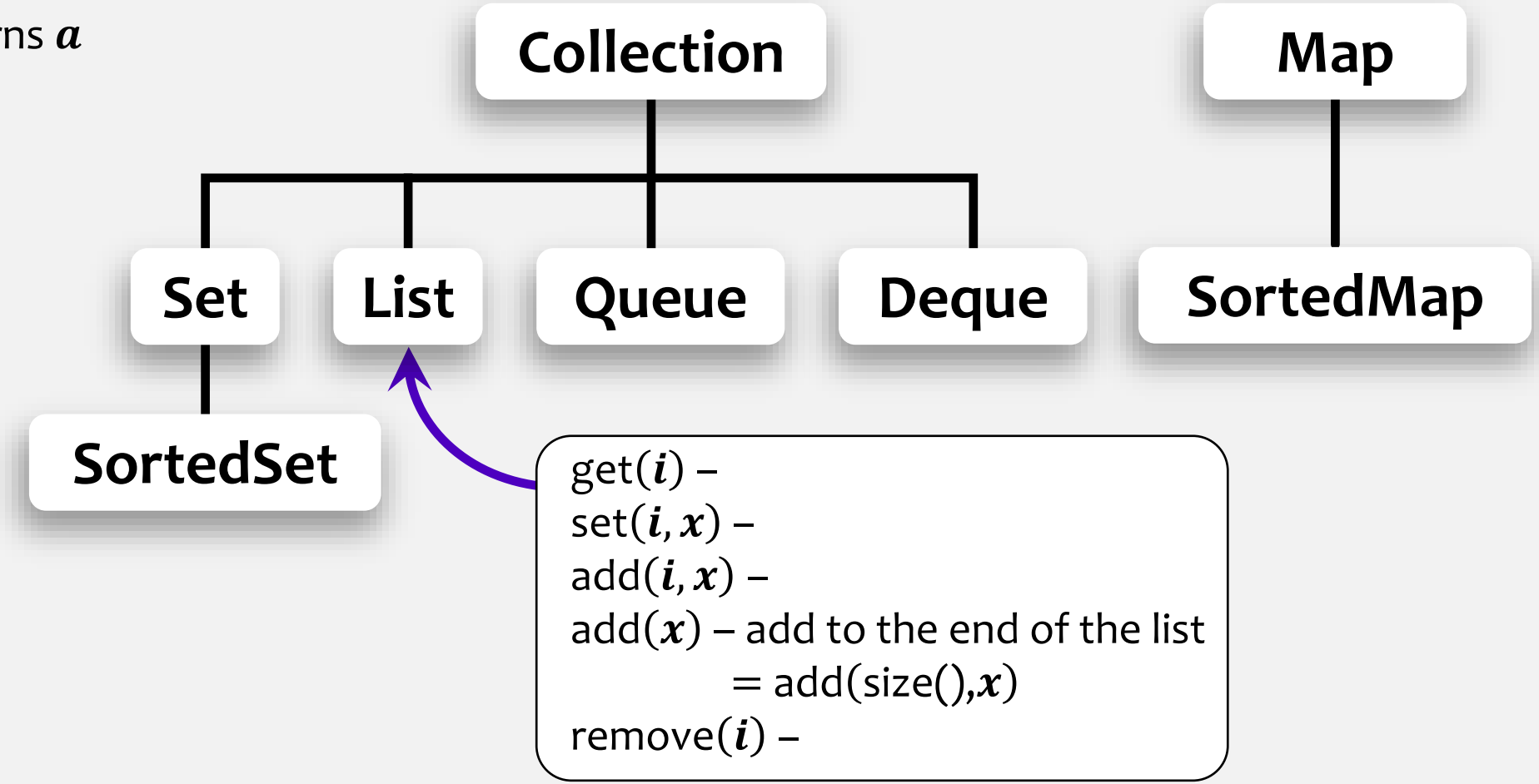
SSet:
Iteration: **2, 3, 5, 8, 9**
first() returns **2**
last() returns **9**
headSet(**6**) returns **2, 3, 5**
headSet(**2**) returns empty set
tailSet(**5**) returns **5, 8, 9**

The Java Collections Framework: Interfaces



List:

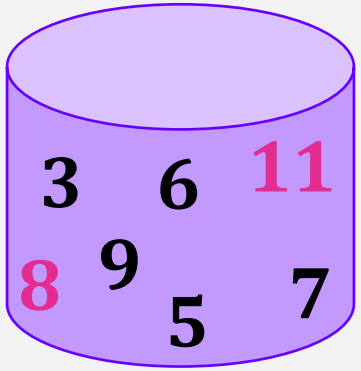
get(4) returns a
set(2, w)
add(6, d)
add(r)
remove(5)



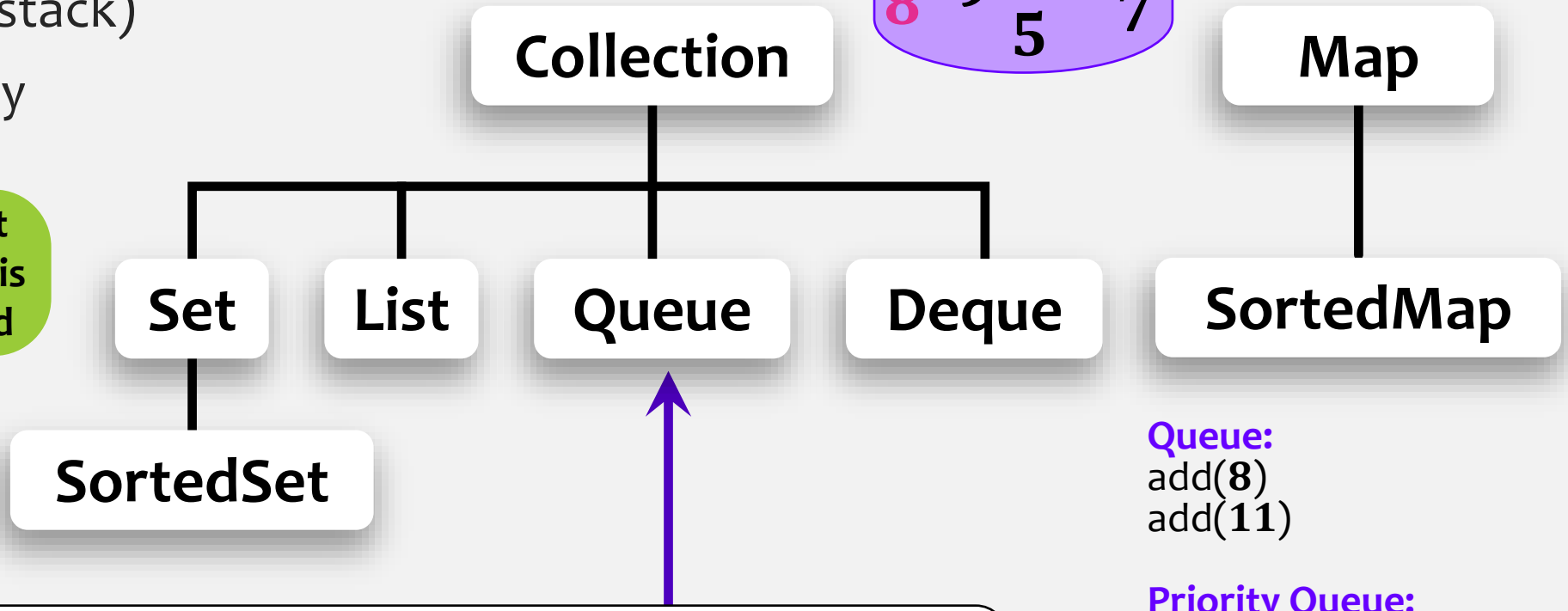
The Java Collections Framework: Interfaces

Three main kinds of queues:

1. FIFO
2. LIFO (stack)
3. Priority



smallest element is removed



add(x) –
remove() – remove one element from the queue and return it.
(the exact element depends on the "**queueing discipline**")

Queue:
add(**8**)
add(**11**)

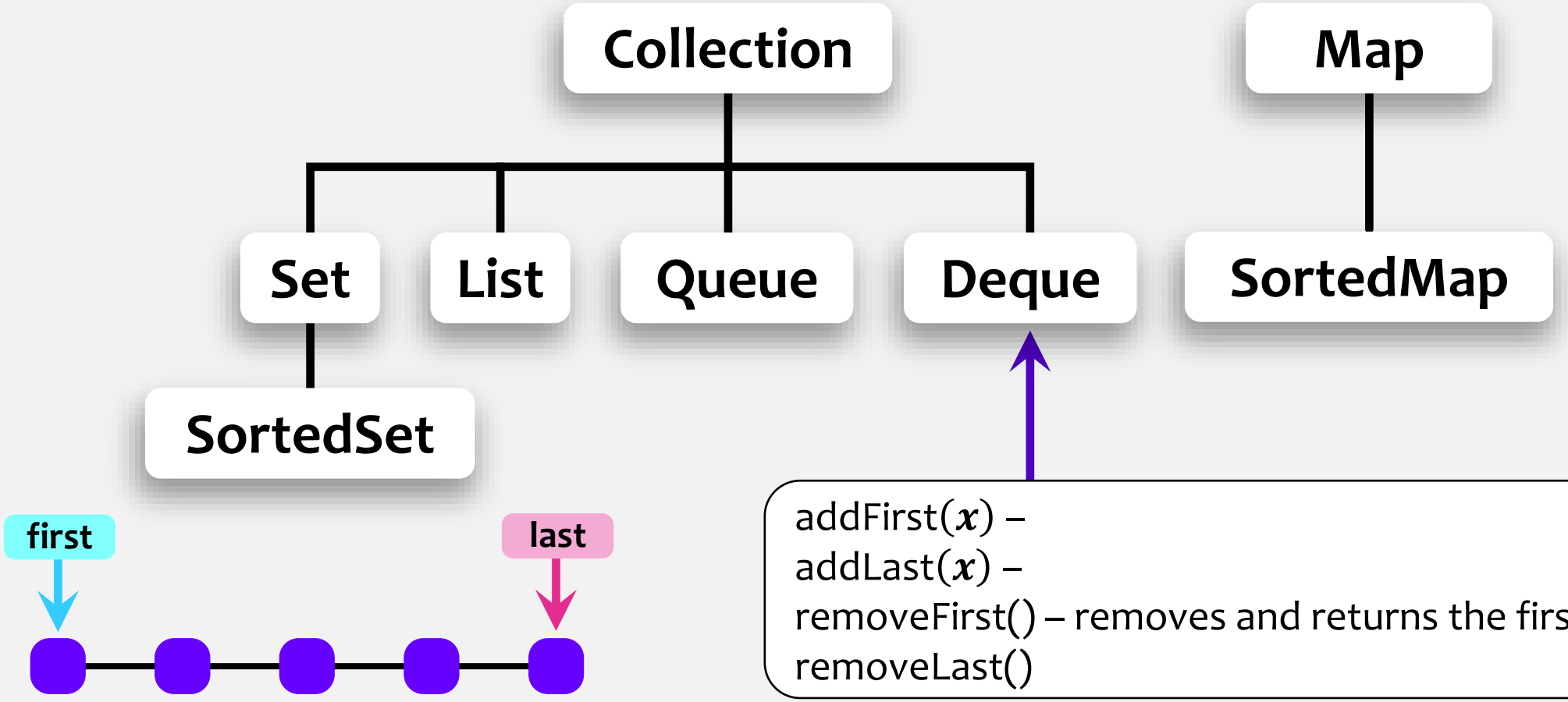
Priority Queue:
remove() – removes/returns **3**

LIFO Queue:
remove() – removes/returns **11**

The Java Collections Framework: Interfaces

Deque – double-ended queue (no indices)

It is a sequence, where you can add and remove to/from either end.



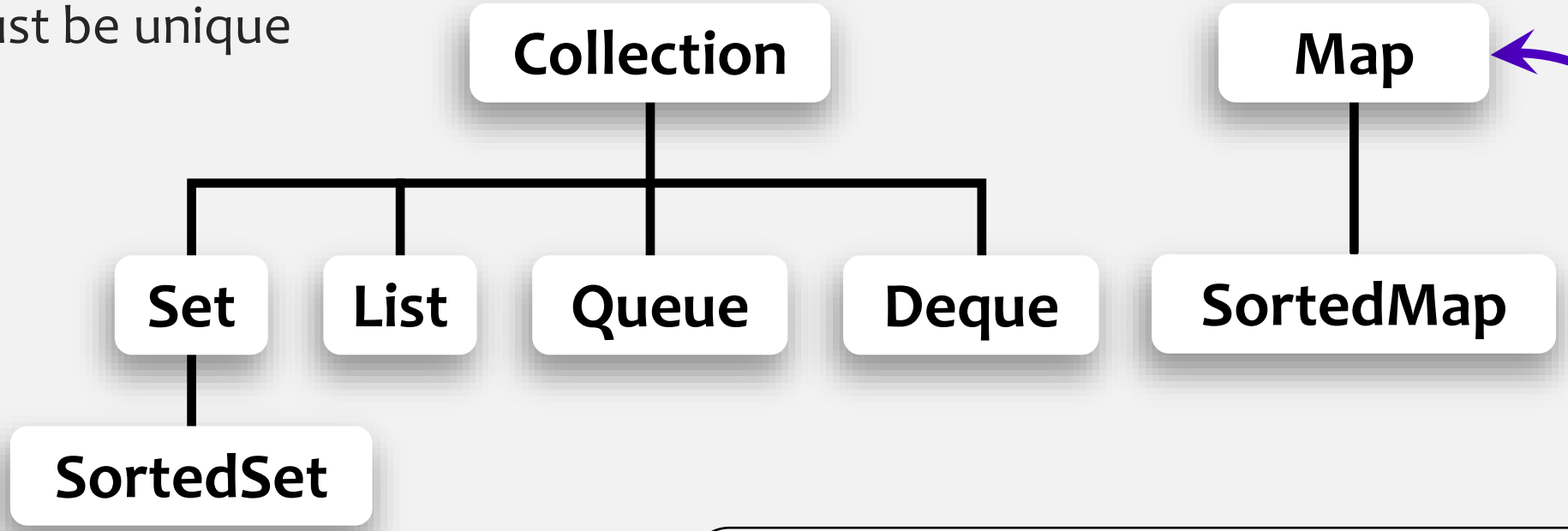
The Java Collections Framework: Interfaces

Map (dictionary)

- maps a set of keys to values
- Keys must be unique

Map:
put("***Mia***", **95**)

Key	Value
"Mia"	~~75~~ **95**
"Ray"	**80**



Sorted Map – a Map in which a key set is a **Sorted Set**

get($\boldsymbol{k}$) – returns the value associated with $\boldsymbol{k}$
put($\boldsymbol{k}, \boldsymbol{v}$) – associates the value $\boldsymbol{v}$ with the key $\boldsymbol{k}$
removeKey($\boldsymbol{k}$) – removes the key $\boldsymbol{k}$ from the key set
containsKey($\boldsymbol{k}$) –

Implementations

Example in Java:

```
List<String> l = new ArrayList<>();
```

Set → HashSet

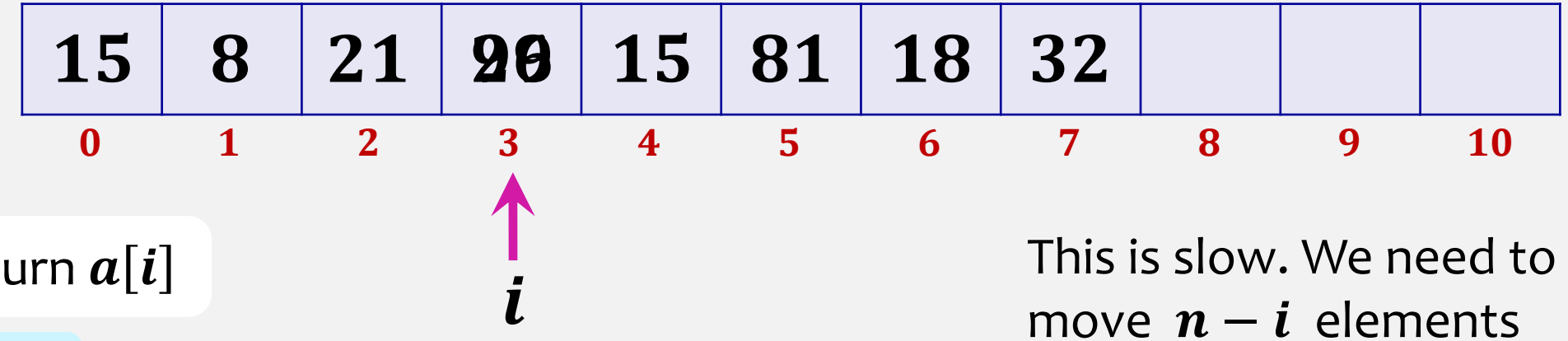
SortedSet → TreeSet

List interface – ArrayList implementation

List can be implemented using single array.

a = new Integer[11]

n = ~~8~~ 9



get(i): return $a[i]$

set(i,x): $y \leftarrow a[i]$
$a[i] \leftarrow x$
return y

$0,..,n-1$

This is slow. We need to move $n - i$ elements

$0,..,n$

add(i,x): takes time $O(n - i + 1)$

time for shifting

This operation is fast only when $i \approx n = \text{size}()$

What if array does not have that one extra position at the back?

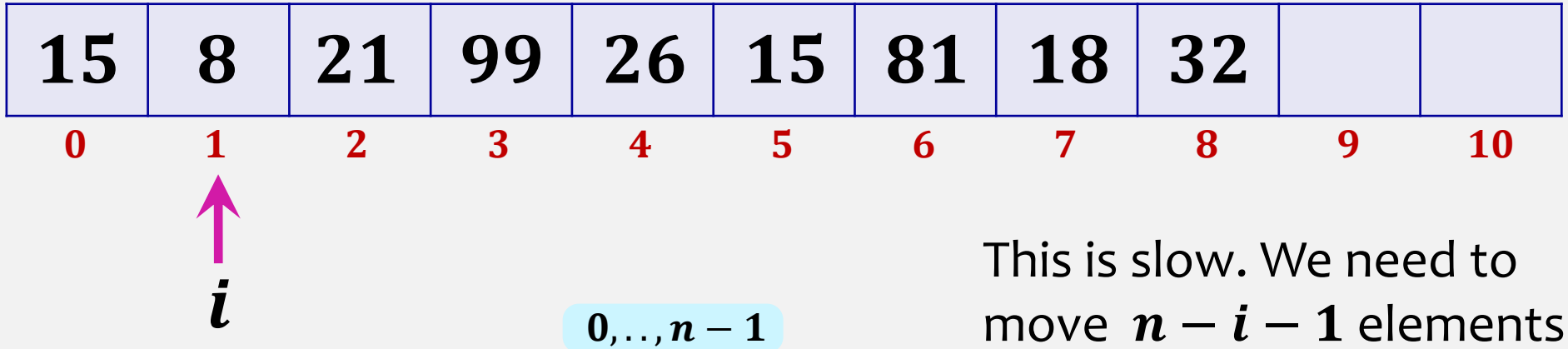
List interface – ArrayList implementation

List can be implemented using single array.

a = new Integer[11]

n = ~~8 9~~ 8

array a:



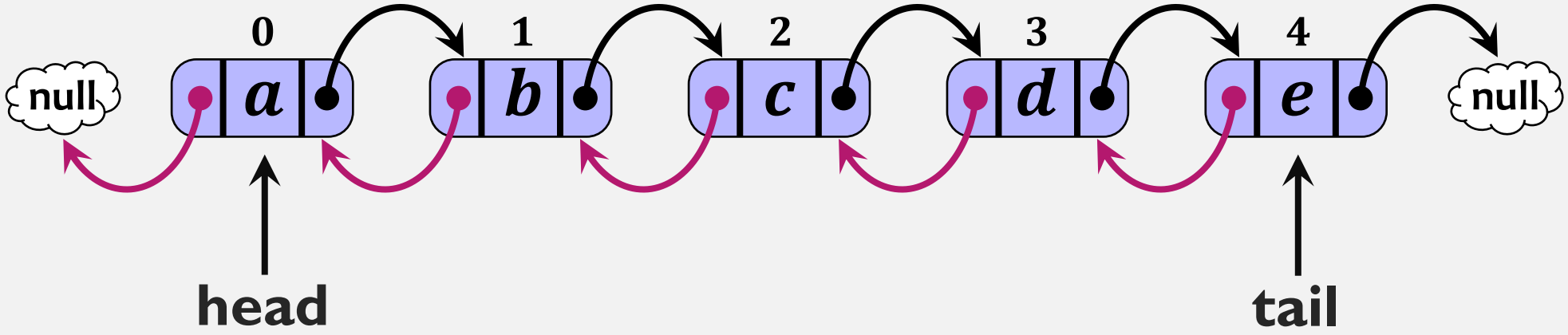
This is slow. We need to move $n - i - 1$ elements

$0, .., n-1$

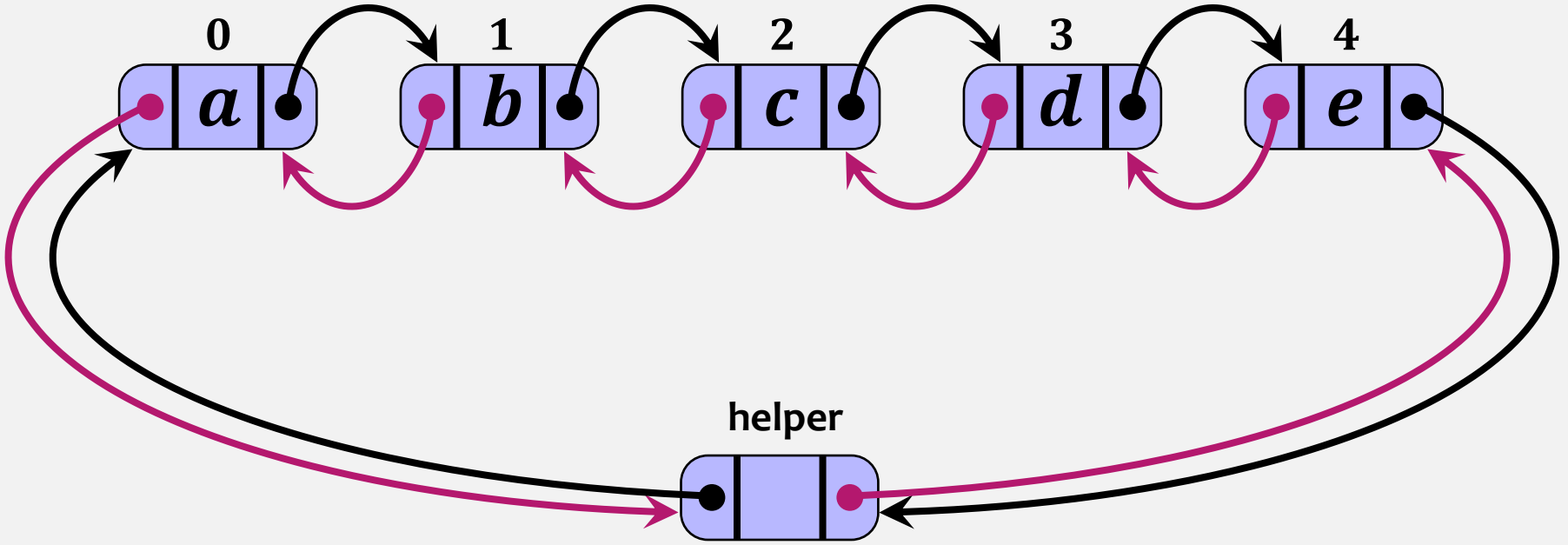
remove(i): takes time $O(n - i + 1)$

This operation is fast only when $i \approx n = \text{size}()$

List interface – LinkedList implementation

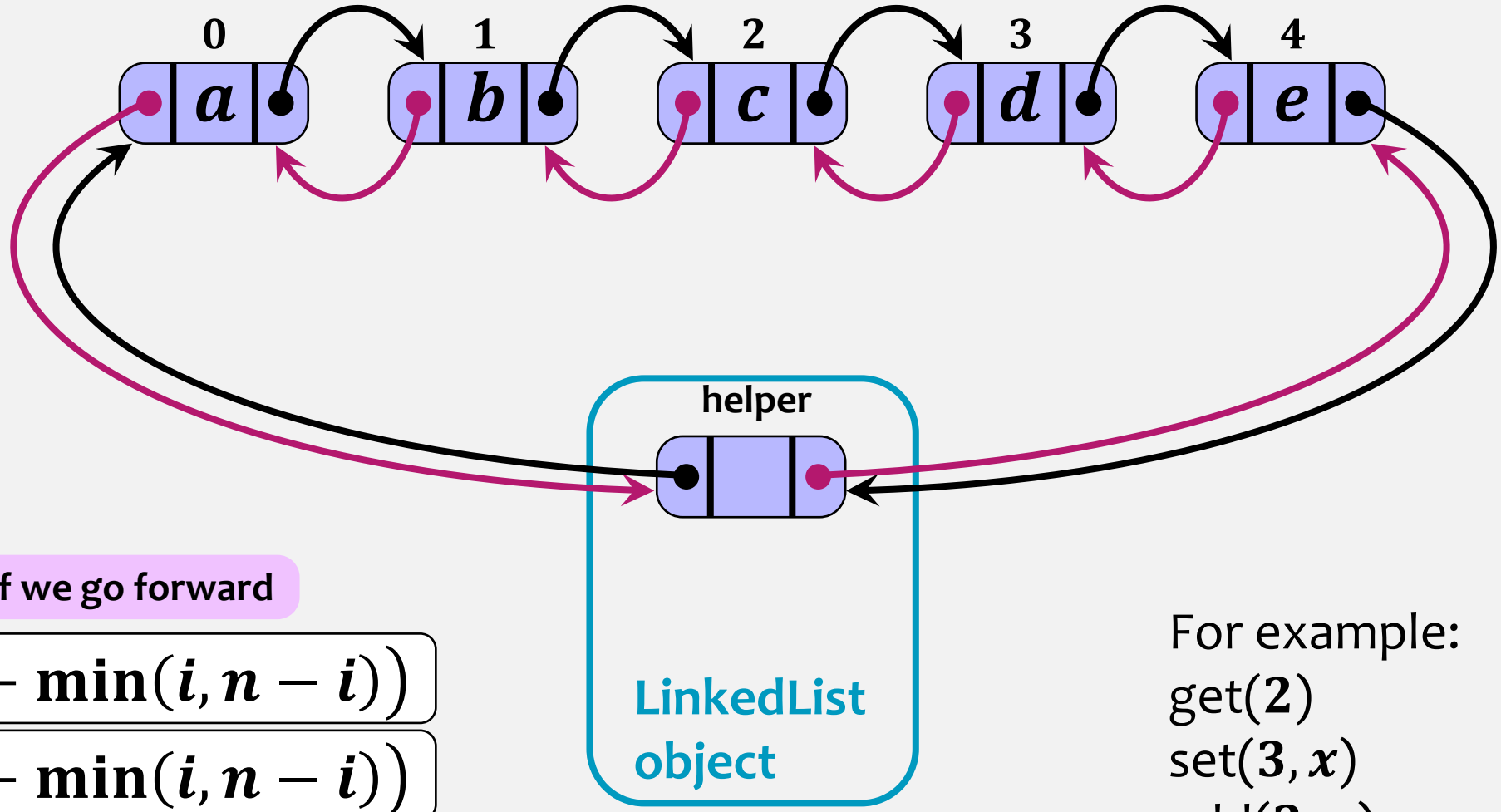


List interface – LinkedList implementation



List interface – LinkedList implementation

indices are implicit



if we go forward

get(i) $O(1 + \min(i, n - i))$

set(i, x) $O(1 + \min(i, n - i))$

add(i, x)

remove(i)

For example:

get(2)

set($3, x$)

add($3, a$)

remove(1)

LinkedList vs ArrayList

add/remove operations:

	LinkedList	ArrayList
	$O(1 + \min(i, n - i))$	$O(n - i + 1)$
If i is small:	**fast**	**slow**

Add/remove operations near each end of the list are **fast** for **LinkedList**

ArrayList is great when you need random access without adding or removing elements.

LinkedList does not give you fast random access, but it is very fast for adding/removing at either end.

How to get started

- Review asymptotic analysis (Big-$\boldsymbol{O}$, $\boldsymbol{\Omega}$, $\boldsymbol{\Theta}$)
- Get familiar with the course's website.
- Read outline of the course and the Calendar
- Introduce yourself in the discussion forum
- Start working on **assignment 1**
- And practice, practice, practice!